Reproducing TransWeave: Jump-Diffusion Transfer Learning for Cryptocurrency Markets

Original paper: Goertzel (2025), *A Unified Framework for Jump-Diffusion Modeling and Transfer Learning in Financial Markets with Behavioral Risk Measures*

Source asset: ETH | **Target assets**: BTC, SOL, DOGE
Data: 15-minute OHLCV + on-chain features, Jan 2022 – Jun 2025
Framework: PyTorch 2.x, Temporal Fusion Transformer, Merton jump-diffusion density

Reproduction report by Tianxiao (Andy) Wang · March 8, 2026

---

1. Introduction & Summary of Results

This report presents a practical implementation of Goertzel (2025), "A Unified Framework for Jump-Diffusion Modeling and Transfer Learning in Financial Markets with Behavioral Risk Measures." The reproduction uses cryptocurrency data: **ETH as the source asset**, transferring to **BTC, SOL, and DOGE** as targets. We use 15-minute OHLCV bars from January 2022 to June 2025, with 14 engineered features including realized volatility, RSI, and on-chain activity z-scores. The three targets were chosen to span a range of transfer difficulty: BTC (high correlation, similar market structure), SOL (high-volatility Layer-1 with distinct ecosystem dynamics), and DOGE (meme-driven, low fundamental correlation).

1.1 Source Model TFT Result (ETH)

Table 1 shows the progressive improvement from a Gaussian baseline (GBM) through the full TFT-JD model on ETH test data (8,736 bars).

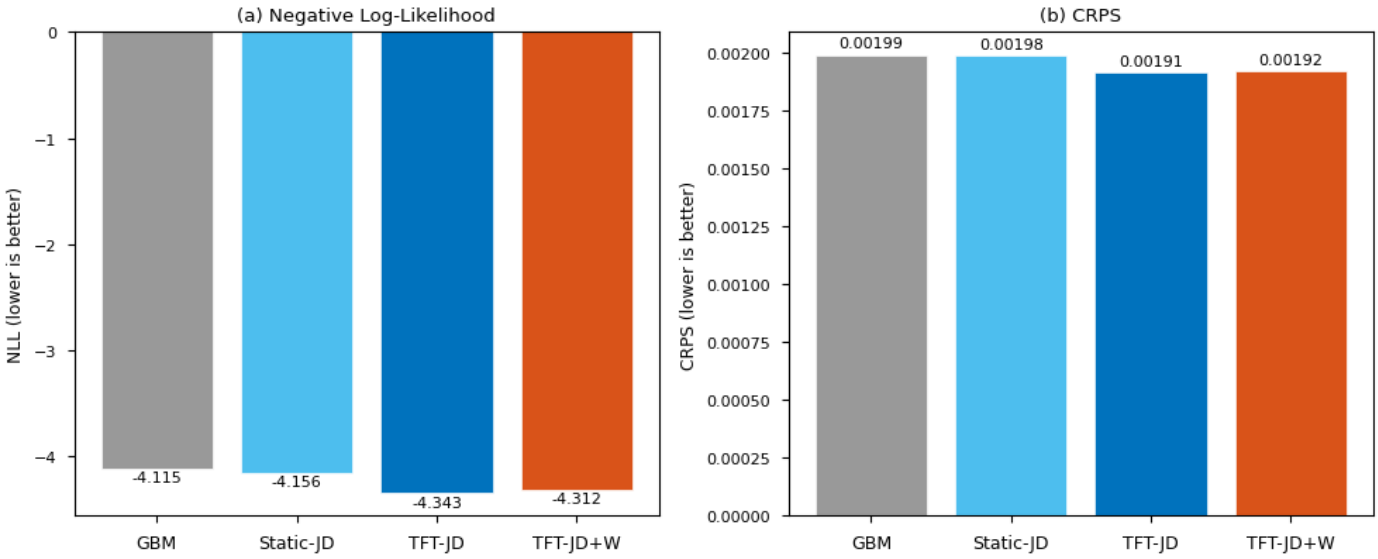
Table 1. ETH test-set Result:

Model	NLL ↓	CRPS ↓	VaR 5% Breach	PIT KS stat ↓
GBM	-4.115	0.00199	—	—
Static-JD	-4.156	0.00198	9.1%	0.045
TFT-JD	-4.343	0.00191	5.6%	0.017
TFT-JD+W	-4.312	0.00192	6.7%	0.043

NLL: negative log-likelihood under JD density (Eq. 8); CRPS: continuous ranked probability score (Eq. 7); VaR breach: fraction of test returns exceeding the 5% VaR threshold (ideal = 5%); PIT KS: Kolmogorov–Smirnov statistic for probability integral transform uniformity (lower = better calibrated). *For continuous PDFs on low-variance returns, density can exceed 1, yielding positive log-likelihood and negative NLL; lower NLL indicates better fit. GBM uses a single static Gaussian; VaR breach and PIT KS were not computed as GBM serves only as a likelihood baseline.*

Two results stand out. First, the jump from GBM to Static-JD is modest (ΔNLL = −0.04), but TFT conditioning adds another −0.19 — time-varying parameters matter far more than the jump component alone. Second, TFT-JD+W (with weakness regularization) slightly underperforms TFT-JD on NLL (−4.312 vs −4.343). This is expected: the weakness term trades raw likelihood for behavioral robustness, a tradeoff that becomes relevant in the transfer setting (§6).

Figure 1: ETH Source Model Ablation



1.2 Transfer Results

Table 2 summarizes the cross-asset transfer experiments. We compare four approaches:

- **Scratch**: A lightweight target-asset JD model (3-layer MLP) trained from random initialization on target data alone, with no source information. This is the "no transfer" baseline.
- **Direct**: Source ETH TFT-JD parameters applied directly to the target asset without any adaptation — testing whether raw parameter sharing works.
- **TransWeave**: ETH parameters mapped through the learned transfer map $T(\theta^{(a)}, \text{regime}) \to \theta^{(b)}$, jointly trained with weakness regularization (Algorithm 1).
- **Target-TFT**: A full TFT-JD model independently trained on the target asset's own data (base model without weakness regularization). This serves as a strong target-only benchmark trained directly on the target asset with the full architecture.

Table 2. Transfer results on test set (single-line format for PDF export):

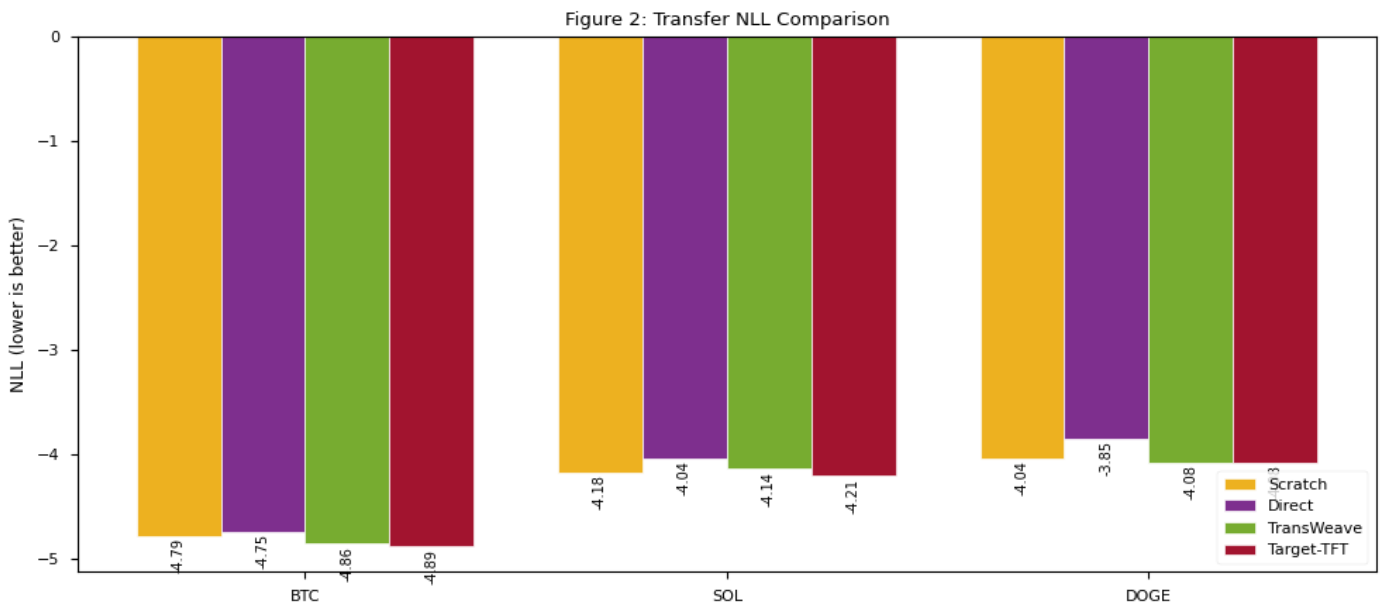
Target	ρ_{regime}	S_{transfer}	Decision	Scratch	Direct	TransWeave	Target-TFT	Δ(TW−Scratch)
BTC	0.768	0.57	Full	-4.788	-4.746	-4.860	-4.888	-0.072
SOL	0.714	0.391	Partial	-4.177	-4.041	-4.142	-4.211	+0.035
DOGE	0.662	0.44	Partial	-4.038	-3.850	-4.084	-4.083	-0.046

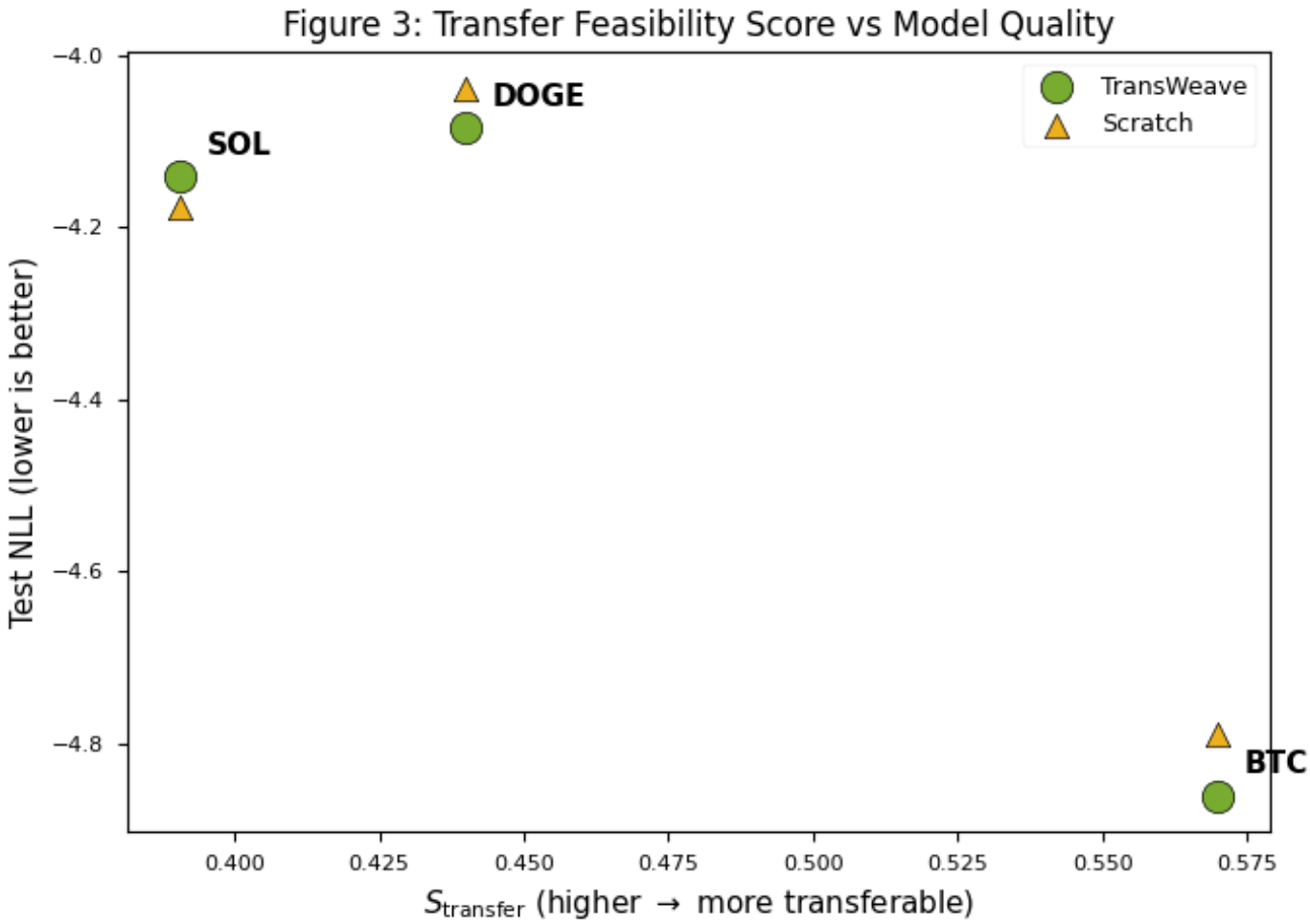
Transfer results on test set. ρ_{regime}: regime overlap between ETH and target (Eq. 13); S_{transfer}: composite feasibility score (Eq. 58); Decision: Full ($\rho \geq 0.7$, $\Delta W_{PT} < 0.1$, $S_{\text{transfer}} \geq 0.5$), Partial (otherwise above reject threshold), Reject ($\rho < 0.3$ or $W_{JD} > W_{\text{crit}}$ or $S_{\text{transfer}} < 0.3$). NLL columns show negative log-likelihood under the JD density (negative values are expected for continuous PDFs; lower NLL = better fit); $\Delta < 0$ means TransWeave outperforms Scratch.

TransWeave improves over Scratch for BTC ($\Delta = -0.072$) and DOGE ($\Delta = -0.046$). SOL is the most challenging target: despite passing the regime correlation threshold ($\rho = 0.714 > 0.7$), its composite $S_{\text{transfer}} = 0.391$ triggers a downgrade from Full to Partial mode. TransWeave slightly underperforms Scratch on SOL (+0.035), consistent with the lowest S_{transfer} among the three targets.

Compared to the target-only benchmark (Target-TFT), TransWeave is closer on BTC ($\Delta = +0.028$) and DOGE ($\Delta = -0.001$, effectively matched within optimization variability), while SOL ($\Delta = +0.069$) remains harder. Notably, DOGE — a meme coin with the lowest regime correlation — is where TransWeave most closely matches the target-specific model, suggesting that the transfer map captures useful cross-asset structure even when regime similarity is low.

Across all targets, TransWeave substantially outperforms Direct transfer (applying ETH parameters without adaptation), indicating that the learned mapping T adds value beyond naive parameter sharing.





Higher S_{transfer} corresponds to better TransWeave performance in this three-target experiment (Figure 3). BTC ($S_{\text{transfer}} = 0.570$) benefits most from transfer, with TransWeave clearly beating Scratch; DOGE ($S_{\text{transfer}} = 0.440$) also sees meaningful improvement; SOL ($S_{\text{transfer}} = 0.391$) is the only case where TransWeave does not surpass Scratch. The composite S_{transfer} score, computed before any training, is directionally predictive of which targets benefit from the TransWeave pipeline.

1.3 Implementation Highlights

Beyond the quantitative results, several observations and design decisions from the reproduction process are worth highlighting:

1. Unconstrained MLE induces λ–σ_J degeneracy. Maximum likelihood estimation of the JD model as well as the TFT model tends to drive jump intensity λ toward extreme large values while shrinking jump size σ_J. We resolve this through a three-step pipeline: first estimate empirical jump frequency using BNS and Lee-Mykland detection, then fit a static JD model via MAP to establish a baseline, and finally incorporate the empirical estimate as a prior on $p_{\text{jump}} = 1 - e^{-\lambda \Delta t}$. This produces more interpretable parameters.

2. Phase 3/4 loss landscape interaction. The TransWeave training has two phases: Phase 3 (joint training of the transfer map T and a weak target model) and Phase 4 (fine-tuning with JD likelihood + weakness signal). We found that an aggressive weakness weight in Phase 3 ($\lambda_{wpt} = 0.5$) pushes W_{PT} to an almost minimum, leaving Phase 4 no room to improve. Reducing λ_{wpt} to 0.1 flattens the landscape and enables Phase 4 to make meaningful updates (§6.3–6.4).

3. Transfer degree and the Partial tier. The degree of transfer is encoded by the Phase 3 λ_{TW} weight: Full uses a larger λ_{TW}, Partial and Weak use smaller values. In our implementation, the "partial" tier from §5 and the "marginal" tier (where $S_{\mathrm{transfer}} < 0.5$ in §6) are merged into a single **Partial** mode. SOL qualifies for Full by $\rho_{\mathrm{regime}} = 0.714$ but is downgraded to Partial because its $S_{\mathrm{transfer}} = 0.391$ falls below 0.5.

4. Parameter-space transfer, not state-space. Our implementation adopts a parameter-space simplification: $T(\theta^{(a)}, \mathrm{regime}) \to \theta^{(b)}$. The map does not condition on features $X^{(a)}$ and $X^{(b)}$, leaving a gap between our implementation and the full theoretical framework (§8).

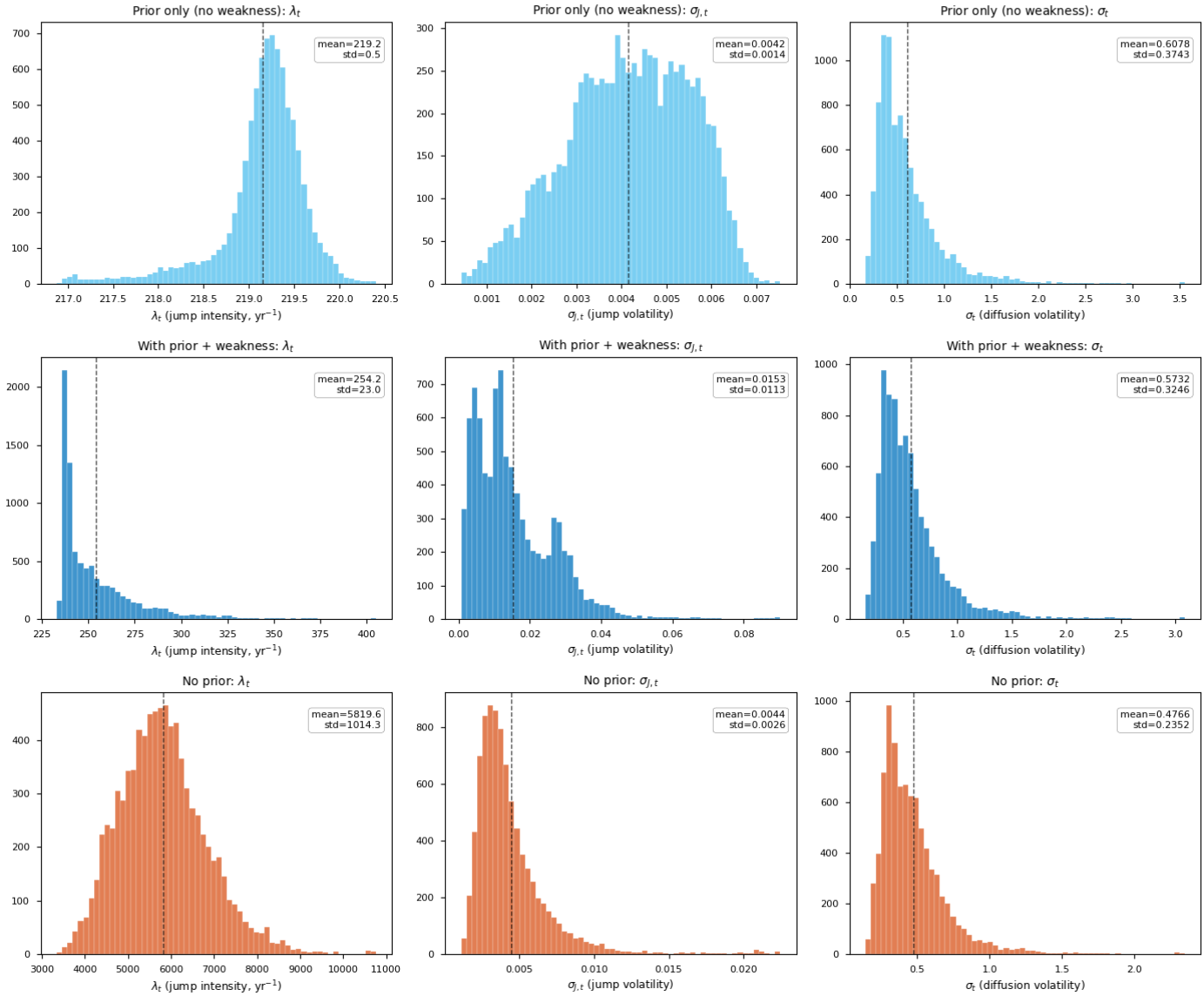
1.4 Jump Frequency Estimation & λ Prior Calibration

We ran both static MLE and TFT-JD training without any λ prior.

Static JD — unconstrained MLE vs MAP with empirical prior:

Setting	λ (yr⁻¹)	σ_J	μ_J	Interpretation
Pure MLE (no prior, no bounds)	9235	0.0058	-0.000108	$\lambda \to \infty$, $\sigma_J \to 0$: jumps degenerate into diffusion
MAP with empirical prior (selected checkpoint)	216	0.0224	-0.00072	Interpretable: ~4 jumps/week, meaningful amplitude

Figure 4: TFT-JD Parameter Distributions — Prior Only / Prior+Weakness / No Prior



Without any prior or parameter bounds, MLE drives λ to over 9000 yr⁻¹ while σ_J collapses to 0.006 — the jump component degenerates into additional diffusion, with each "jump" contributing negligible amplitude. The MAP prior with empirical p_center produces λ = 216 with σ_J = 0.022, corresponding to roughly 4 jumps per week with meaningful size.

Figure 4 shows the same phenomenon in the TFT-JD neural setting. With the selected source checkpoint (prior + weakness fine-tuning), λ_t has mean 254.2 (std 23.0), $\sigma_{J,t}$ mean 0.0153, and σ_t mean 0.5732. Without the prior, the corresponding values shift to λ_t mean 5819.6 (std 1014.3), $\sigma_{J,t}$ mean 0.0044, and σ_t mean 0.4766 — consistent with a degenerate many-small-jumps regime. Notably, using only the prior yields output λ_t very close to the prior mean with small $\sigma_{J,t}$, whereas the weakness-finetuned version exhibits greater variability in λ_t and more reasonable $\sigma_{J,t}$ (§4.3).

Empirical jump frequency estimation. Rather than picking λ_center by hand, we estimate it from data using two complementary detectors:

- **Barndorff-Nielsen & Shephard (BNS)** test: compares realized variance with bipower variation to detect days with significant jump activity. Applied to ETH training data (2022-01 – 2024-09), BNS flags 826 jump-days out of ~3.5 years, giving $\hat{\lambda}$_BNS ≈ 236 yr^{-1}.
- **Lee & Mykland** test: a bar-level jump detector using the ratio of absolute return to local bipower volatility. With a 5% significance threshold, it identifies 156 jumps, giving $\hat{\lambda}$_LM ≈ 45 yr^{-1}.

The two estimates bracket the true frequency: BNS is liberal (counts any day with excess variation), while Lee-Mykland is conservative (only flags extreme individual bars). We combine them with a threshold-based cross-check (|r| > kσ_rolling for k ∈ {3, 4, 5}), yielding a recommended $\hat{\lambda}$ ≈ 140 yr^{-1}.

MAP prior formulation. The prior is placed on the per-bar jump probability p = 1 – exp(–λΔt):

$$\log \pi(p) = -w_{\text{prior}} \cdot \left(\text{logit}(p) - \text{logit}(p_{\text{center}})\right)^2$$

where p_center is derived from $\hat{\lambda}$. This logit-normal form concentrates penalty near the center while allowing the optimizer freedom in the tails. The selected source checkpoint uses p_center from the empirical pipeline, producing λ = 216 with σ_J = 0.022 — a physically interpretable parameterization where jumps occur roughly 4 times per week with meaningful amplitude.

---

2. Data & Experimental Setup

2.1 Asset Selection

We choose **ETH as the source asset** for three reasons: it has the second-largest market cap in crypto (ensuring deep liquidity), a rich on-chain data ecosystem accessible via Dune Analytics, and a well-documented history of both diffusion-regime and jump-regime behavior (the 2022 LUNA/FTX crashes, 2024 ETF-driven rally).

The three target assets are selected to span a gradient of transfer difficulty:

Target	Role	Expected Transfer Difficulty
BTC	Large-cap, highest ρ with ETH	Easy — similar macro-driven dynamics, correlated regime transitions
SOL	High-volatility Layer-1	Medium — shares crypto beta but has distinct ecosystem shocks (e.g., Solana-specific outages, DeFi activity spikes)
DOGE	Meme coin, sentiment-driven	Hard — low fundamental correlation, jump structure driven by social media events rather than on-chain fundamentals

This design ensures the transfer framework is tested across a meaningful range: if TransWeave only works on BTC (the "easy" case) or it also helps on DOGE.

2.2 Feature Engineering

We use **15-minute bars** as the core frequency for two reasons: (i) it is enough to yield sufficient data for training the TFT (96 bars per 24 hours); (ii) it is coarse enough so that on-chain and calendar covariates align well — at shorter intervals, hourly on-chain metrics and day-of-week encoding would appear almost static, with little variation across bars.

The TFT encoder takes two inputs:

Historical features X_t — a window of 96 bars (= 24 hours at 15-minute resolution), with 14 features per bar:

#	Feature	Source	Window
1	`log_return`	OHLCV	—
2–4	`realized_vol_1h / 6h / 24h`	Rolling std of log returns	4 / 24 / 96 bars
5	`volume_log_change`	log(vol*t* / *vol*{t-1})	—
6	`rsi_14`	14-period RSI	14 bars
7	`bb_width_20`	Bollinger Band width	20 bars
8	`vol_of_vol`	Rolling std of `realized_vol_1h`	24 bars
9	`return_skew_24h`	Rolling skewness	96 bars
10	`high_low_range`	log(high/low) per bar	—
11–14	`active_addresses_z`, `gas_fee_z`, `whale_transfers_z`, `dex_volume_z`	Dune on-chain, hourly, z-scored	168h rolling

Features 1–10 are computed from 15-minute OHLCV data. Features 11–14 are on-chain metrics fetched via Dune Analytics at hourly granularity, forward-filled to 15-minute bars (so four consecutive bars within the same hour share the same on-chain value), and z-scored with a 168-hour (1-week) rolling window.

Known future covariates Z_t — 4-dimensional time encoding at the prediction timestamp:

- `hour_sin`, `hour_cos`: sinusoidal encoding of hour-of-day
- `dow_sin`, `dow_cos`: sinusoidal encoding of day-of-week

These are deterministic calendar features known at prediction time — no future information leaks.

2.3 Data Split & Leakage Audit

Segment	Period	Bars (approx.)	Purpose
Train	2022-01-01 – 2024-09-30	~96,000	Model fitting, HMM regime detection, feature standardization
Validation	2024-10-01 – 2025-03-31	~17,500	Early stopping, hyperparameter selection
Test	2025-04-01 – 2025-06-30	~8,700	All reported metrics

The train period covers a bear–bull cycle (2022 LUNA/FTX crash through 2024 BTC ETF rally), providing diverse regime exposure. The test period is a strict hold-out that was never used for any model selection.

Leakage audit. An independent audit (`scripts/run_leakage_audit.py`) checked five potential leakages and made sure: (1) HMM fitted on train data only, (2) all rolling features use causal backward-looking windows, (3) standardization scopes documented and train-only where applicable, (4) Z_{future} contains only deterministic calendar features, (5) X_{hist} window does not extend past the prediction boundary. All five checks passed with zero critical issues.

---

3. Source Model: JD Density & TFT Conditioning

3.1 Jump-Diffusion Density

The return process follows Merton's jump-diffusion (Eq. 1):

$$r_{t+1} = \mu_t \Delta t + \sigma_t \sqrt{\Delta t}\, \varepsilon_t + \sum_{k=1}^{N_t} Y_{t,k}$$

where $\varepsilon_t \sim \mathcal{N}(0,1)$, $N_t \sim \text{Poisson}(\lambda_t \Delta t)$, and $Y_{t,k} \sim \mathcal{N}(\mu_J, \sigma_J^2)$. All parameters are annualized; $\Delta t = 1/35040$ for 15-minute bars.

The log-density (Eq. 8) is a truncated Poisson mixture of Gaussians:

$$\log p(r \mid \theta) = \log \sum_{n=0}^{n_{\max}} \frac{(\lambda \Delta t)^n e^{-\lambda \Delta t}}{n!} \cdot \phi\big(r;\, \mu \Delta t + n\mu_J,\, \sigma^2 \Delta t + n\sigma_J^2\big)$$

We truncate at $n_{\max} = 10$. The sum is computed in log-space via `logsumexp` to avoid numerical underflow — critical when $\lambda \Delta t$ is small and most Poisson terms are near zero.

Static MAP estimation. Before training the TFT, we fit a static 5-parameter JD model $\theta_{\text{static}} = (\mu, \sigma, \lambda, \mu_J, \sigma_J)$ to ETH training returns via MAP (§2.4). This serves two purposes: (i) as an integration test for the density implementation, and (ii) as the initialization anchor for TFT-JD parameter heads.

Asset	μ	σ	λ (yr⁻¹)	μ_J	σ_J
ETH	0.6	0.453	216	-0.00072	0.0224
BTC	0.6	0.371	202	-0.00052	0.02
SOL	0.32	0.834	159	0	0.0425
DOGE	0.21	0.649	207	-0.00036	0.0324

Table 3. Static JD MAP parameters (annualized). SOL has the highest σ (0.834) and lowest λ (159), consistent with its more volatile, less jump-dominated dynamics. BTC and ETH are similar in structure, supporting the transfer feasibility between them. Most μ_J values are slightly negative, reflecting the asymmetry of crypto jumps; SOL is approximately zero.

3.2 TFT-JD Architecture

The Temporal Fusion Transformer encoder (Eq. 2–6) maps historical features and time covariates to a hidden representation, from which five independent parameter heads produce time-varying JD parameters:

Head	Output	Constraint	Activation
f_μ	μ_t	Unconstrained	Linear
f_σ	σ_t	> 0	Softplus + ϵ
f_λ	λ_t	> 0, bounded	Softplus + ϵ, clamp $\leq \lambda_{\max}$
f_{μ_J}	$\mu_{J,t}$	$\in [-0.06, 0.06]$	Tanh$\times$scale
f_{σ_J}	$\sigma_{J,t}$	> 0	Softplus + ϵ

Each head is a small MLP (Linear → ReLU → Linear). The output biases are initialized from the static MAP parameters (Table 3), so the TFT starts near a known-good solution and learns time-varying deviations. Weight initialization uses small random values ($\text{std} = 10^{-3}$) to ensure the initial output is close to the bias.

Architecture hyperparameters: hidden size = 64, attention heads = 4, dropout = 0.15, $\epsilon = 10^{-4}$. Input shape: $X_{\text{hist}} \in \mathbb{R}^{B\times 96\times 14}$, $Z_{\text{future}} \in \mathbb{R}^{B\times 1\times 4}$.

3.3 Training Loss

The combined loss follows Eq. (7):

$$L(\theta, r) = \text{NLL}(\theta, r) + \alpha \cdot \text{CRPS}(\theta, r) + \beta \cdot R(\theta)$$

- **NLL**: $-\sum_t \log p(r_t \mid \theta_t)$ using the JD density (Eq. 8)
- **CRPS**: Continuous Ranked Probability Score, estimated via Monte Carlo sampling ($M = 200$ draws from $p(r \mid \theta)$). The JD mixture has no closed-form CRPS, so MC approximation is necessary.
- $R(\theta)$: A soft λ-prior regularizer (`lambda_prior_weight` $= 0.05$) that penalizes deviation of λ_t from the static MAP center, extending the §2.4 prior into the neural setting.

Training uses Adam with `lr = 3e-4`, weight decay 10^{-3}, gradient clipping at 1.0, batch size 128, and early stopping on validation NLL with patience 15. Three training modes are used:

- **Base**: NLL + CRPS only
- **Scratch+W**: NLL + CRPS + weakness regularization from random init
- **Finetune+W**: NLL + CRPS + weakness, initialized from base checkpoint

3.4 TFT Discussion

Returning to Table 1, the progressive improvement tells a clear story:

GBM → Static-JD (ΔNLL = −0.04). Adding the jump component provides a modest but consistent improvement. The static JD captures heavy tails better than a Gaussian, but with fixed parameters it cannot adapt to changing market conditions — hence the limited gain.

Static-JD → TFT-JD (ΔNLL = −0.19). This is the largest single improvement. Time-varying parameters allow the model to increase σ_t during volatile periods and adjust λ_t when jump activity changes. The 5× larger gain over the static jump component confirms that **parameterization dynamics matter more than the jump specification itself**.

TFT-JD → TFT-JD+W (ΔNLL = +0.03). Weakness regularization slightly hurts raw likelihood — it diverts some model capacity toward behavioral calibration. However, this tradeoff is by design: the weakness term (§4) encourages the model to produce parameters that are "robust" in a prospect-theoretic sense, which becomes valuable during transfer learning (§6).

PIT calibration. The TFT-JD achieves the best PIT KS statistic (0.017), but with $n = 8{,}736$ test samples even this small deviation produces $p = 0.012$ (p-value computed via `scipy.stats.kstest` using the asymptotic Kolmogorov distribution), formally rejecting uniformity.

---

4. Behavioral Weakness Regularization

4.1 W_{PT} Implementation

Our implementation of W_{PT} (Eq. 37) follows an 8-step computation in `src/behavioral/weakness.py::compute_w_pt`:

Step	Operation	Code
1	Jump probability	$p_{\text{jump}} = 1 - \exp(-\lambda_t \cdot \Delta t)$, clamped to [1e-10, 1]
2	Prelec weighting	$\pi_{\text{val}} = \exp(-(-\ln p_{\text{jump}})^{\delta})$ with $\delta = 0.69$
3	Prospect value	$m = \mu_t - \lambda_t \cdot \mu_{J,t}$; $v_{\text{val}} = m^{\alpha}$ (gain) or $-\lambda_{\text{loss}}(-m)^{\beta}$ (loss)
4–5	Tail risk	CVaR at $\alpha = 5\%$ under JD density; $\text{risk_factor} = \exp(-\gamma \cdot \text{CVaR}_{\text{loss}})$ with $\gamma = 7.0$
6	Raw score	$\text{raw} = \pi_{\text{val}} \times v_{\text{val}} \times \text{risk_factor}$
7	Activation	$w_{\text{pt_raw}} = \text{softplus}(\text{raw})$
8	Normalization	$w_{\text{pt}} = w_{\text{pt_raw}} / \max(w_{\text{pt_raw}}).\text{detach}()$, clamped to [1e-8, 1]

Step 8 uses **batch-max normalization**: dividing by the maximum raw score within the current batch (detached from the gradient graph). This keeps $W_{PT} \in (0, 1]$ and ensures the $-\log W_{PT}$ loss term stays bounded. In Phase 3 (§6.3), source and target samples are normalized jointly within the same batch — this is critical for cross-asset comparability.

CVaR computation uses two modes: an analytic heuristic (fast, differentiable, used during training) and Monte Carlo sampling with 5000 draws (used during validation for accuracy). The analytic version approximates the JD mixture's left tail using the dominant Gaussian component weighted by Poisson probabilities — accurate when λΔt is small.

4.2 Weakness Loss Design

The weakness loss L_{weakness} plays different roles in the two TransWeave training phases:

Phase 3 (joint training, §6.3) uses the full formula (Eq. 50):

$$L_{\text{weakness}} = -\log W_{PT} + \text{MSE}(W_{PT}^{\text{pred}},\, W_{PT}^{\text{target}})$$

The first term encourages behavioral robustness; the second aligns the transfer map's output weakness profile with the weak target model. Phase 3 weight: $\lambda_{wpt} = 0.1$.

Phase 4 (fine-tuning, §6.4) uses only the log term, following Algorithm 1's gradient form $\nabla L = \nabla L_{\text{data}} - \beta \nabla \log W_{PT}$:

$$L_{\text{weakness}}^{(\text{P4})} = -\log W_{PT}$$

Phase 4 weight: $\lambda_{wpt} = 1.0$ (higher than Phase 3).

4.3 Effect on λ Parameter Dynamics

An unexpected benefit of weakness regularization is its interaction with the λ–σ_J degeneracy (§1.4). We compare three ETH TFT-JD training modes on the same architecture:

Training mode	λ mean	λ std	σ_J mean	Note
Base (prior only)	219	0.5	0.0042	λ locked near MAP center, σ_J collapsed
W-finetune (prior + weakness)	254	23.0	0.0153	λ freed, σ_J interpretable
W-scratch (weakness, no base init)	3334	75.7	0.0370	λ degenerates

W-scratch includes the λ prior (weight=0.05) but the weakness gradient overwhelms it, pulling λ to a degenerate basin before the prior can anchor it.

The base model with prior alone locks λ near 219 with negligible variation — the prior prevents degeneration but also suppresses time-varying signal. The **weakness-finetuned** model (initialized from the base checkpoint, then fine-tuned with weakness loss) breaks this stasis: λ_t acquires meaningful temporal variation (std = 23) while staying in a physically reasonable range, and σ_J increases 3.6× to 0.015.

The potential mechanism: the weakness loss penalizes parameterizations where π(p_jump) · v(m) produces extreme values. Degenerate "many tiny jumps" configurations have high p_jump (pushing π toward 1) but trivial v(m) (near-zero drift adjustment), creating a gradient that pushes the model away from the degenerate region. This only works when starting from a well-initialized base — training from scratch with weakness (W-scratch) cannot find the right basin and degenerates to λ ≈ 3300.

The selected source checkpoint used for all transfer experiments in §6 is the W-finetune variant.

---

5. Regime Detection & Transfer Feasibility

Before running the transfer learning pipeline, we assess whether transfer from ETH to each target is feasible. This follows the paper's two-phase decision process (Algorithm 1, Phases 1–2): first check whether the source and target are in the "same domain" (Phase 1), then determine the transfer mode — Full, Partial, or Weak (Phase 2).

5.1 HMM Regime Detection

We fit a 4-state Gaussian HMM to each asset's feature space to identify market regimes. The number of states is fixed at 4 (not selected via BIC) because the transfer metrics (Eq. 13, 22) require consistent dimensionality across assets.

Feature preparation: The 14 TFT features are downsampled from 15-minute to 1-hour resolution (factor 4), then columns with >50% missing values are dropped. PCA is applied to the remaining features, retaining components explaining ≥85% variance (up to 6 components). This produces a compact, comparable feature space across all four assets.

HMM fitting: Each asset's PCA features are fitted independently with `GaussianHMM(n_components=4, covariance_type="full")`, using 5 random initializations and selecting the best by log-likelihood. The resulting state sequences, transition matrices P, and eigenvector decompositions V are saved for the transfer metrics.

5.2 W_{crit} and Phase 1 (Go/No-Go)

Phase 1 checks whether the JD parameter distributions of source and target are close enough to justify transfer. The Wasserstein distance between rolling-window JD parameter distributions (Eq. 12) is compared against a critical threshold:

$$W_{\text{crit}} = 2\sqrt{(\bar{\sigma}^2 + \bar{\lambda})}$$

where $\bar{\sigma}$ and $\bar{\lambda}$ are rolling-window means over aligned source and target θ sequences. With 30-day non-overlapping windows (2880 bars each, 33 windows total), we compute the p90 of per-window W_{JD} values and check W_{JD}/W_{crit}. W_{crit} is computed per pair; the resulting values are numerically very similar across the three crypto targets (~23.8), reflecting the similar parameter scales of the four assets.

Table 4. Transfer feasibility metrics (single-line format for PDF export):

Target	W_{JD} (p90)	W_{crit}	W_{JD}/W_{crit}	Phase 1	ρ_{regime}	ΔW_{PT} (p90)	S_{transfer}	Decision
BTC	2.247	23.8	0.094	Pass	0.768	0.0409	0.570	Full
SOL	4.721	23.8	0.198	Pass	0.714	0.0810	0.391	Partial
DOGE	3.918	23.8	0.165	Pass	0.662	0.0490	0.440	Partial

Table 4. Transfer feasibility metrics. W_JD/W_crit < 1 passes Phase 1 (same-domain check). All three targets pass comfortably — W_JD/W_crit ranges from 0.094 (BTC) to 0.198 (SOL), far below the rejection threshold. This is perhaps unsurprising: all four assets are cryptocurrencies sharing the same macro environment, so their JD parameter distributions overlap substantially. The Phase 1 check might be more discriminating for cross-asset-class transfer (e.g., crypto → equities).

5.3 ρ_{regime}: Regime Overlap

The regime overlap coefficient ρ_{regime} (Eq. 13) measures how well the HMM eigenvectors of the source align with those of the target:

$$\rho_{\text{regime}} = \max_{\pi} \sum_{i,j} \pi_{ij} \langle v_i^{(a)}, v_j^{(b)} \rangle$$

where the maximization is over permutation matrices π (accounting for arbitrary state labeling). Following the paper, we exclude the stationary eigenvector (eigenvalue λ = 1) from the computation, focusing on the *dynamic* regime structure rather than the equilibrium distribution.

Results: BTC (ρ = 0.768) > SOL (ρ = 0.714) > DOGE (ρ = 0.662). This ordering matches intuition — BTC shares the most market-structure similarity with ETH, while DOGE's meme-driven dynamics produce the most distinct regime transitions.

5.4 S_{transfer}: Composite Feasibility Score

The composite transfer score (Eq. 58) combines all three signals:

$$S_{\text{transfer}} = \exp(-W_{JD}/W_{\text{crit}}) \cdot \rho \cdot \exp(-|\Delta W_{PT}|/0.2)$$

This multiplicative form means each factor can independently suppress the score: high W_JD (parameter distance), low ρ (regime mismatch), or large ΔW_PT (behavioral divergence) all reduce S. The results — BTC (0.570) > DOGE (0.440) > SOL (0.391) — match the observed ranking in this three-target experiment (Table 2).

5.5 Decision Matrix

The paper's Theorem 5.1 specifies three necessary conditions for transfer success: (i) $W_{JD} < W_{\text{crit}}$ (statistical compatibility), (ii) $\rho_{\text{regime}} > 0.5$ (structure alignment), and (iii) $\Delta W_{PT}/(W_{PT}^a + W_{PT}^b) < 0.2$ (behavioral consistency). Phase 2 refines these into a transfer mode assignment:

Condition	Decision
$\rho \geq 0.7$ and $\Delta W_{PT} < \delta_{\text{full}}$ and $S_{\text{transfer}} \geq 0.5$	**Full**
$\rho \geq 0.7$ and $\Delta W_{PT} < \delta_{\text{full}}$ but $S_{\text{transfer}} < 0.5$	**Partial** (downgrade)
$0.5 \leq \rho < 0.7$	**Partial**
$0.3 \leq \rho < 0.5$	**Weak**
$\rho < 0.3$ or $W_{JD} > W_{\text{crit}}$ or $S_{\text{transfer}} < 0.3$	**Reject**

Our implementation uses $\delta_{\text{full}} = 0.1$ (stricter than the paper's theoretical bound of 0.2). When ρ and ΔW_{PT} both pass the Full thresholds but $S_{\text{transfer}} < 0.5$, the decision is downgraded to Partial — this follows Section 6.1 of the paper, which recommends $S_{\text{transfer}} > 0.5$ for full transfer.

BTC ($\rho = 0.768$, $\Delta W_{PT} = 0.041$, $S_{\text{transfer}} = 0.570$): passes all Full thresholds → **Full transfer**. The transfer map operates with the highest λ_{TW} weight (0.1), giving maximum intertwining regularization.

SOL ($\rho = 0.714$, $\Delta W_{PT} = 0.081$, $S_{\text{transfer}} = 0.391$): ρ passes the Full threshold (0.714 > 0.7) and $\Delta W_{PT} < 0.1$, but $S_{\text{transfer}} = 0.391$ falls below 0.5, triggering a **downgrade to Partial**. This is the most instructive case: ρ alone would suggest Full transfer, but the lower S_{transfer} — driven by SOL's higher W_{JD} (0.198 vs BTC's 0.094) and larger behavioral divergence — correctly identifies SOL as a harder target. The Partial mode uses a smaller λ_{TW} (0.05 vs 0.1 for Full).

DOGE ($\rho = 0.662$, $S_{\text{transfer}} = 0.440$): ρ falls in the Partial range (0.5 < 0.662 < 0.7) → **Partial transfer** directly. Despite being the "hardest" target by ρ, DOGE ends up with a higher S_{transfer} than SOL (0.440 vs 0.391), and indeed outperforms SOL in the final transfer results (§1.2).

---

6. TransWeave Transfer Learning

This section details the core contribution: learning a parameter-space transfer map $T(\theta^{(a)}, \text{regime}) \to \theta^{(b)}$ that adapts the ETH source model's JD parameters to each target asset. Training proceeds in two phases: Phase 3 (joint learning of T with a weak target model) and Phase 4 (fine-tuning T on target returns with weakness regularization).

6.1 Architecture

Transfer map T. A regime-conditioned 3-layer MLP:

$$
\begin{aligned}
\text{Input: } & [\theta^{(a)} \in \mathbb{R}^5 \parallel \text{regime}_{\text{onehot}} \in \mathbb{R}^4] \\
\downarrow \quad & \text{Linear}(9, 64) \to \text{ReLU} \\
\downarrow \quad & \text{Linear}(64, 64) \to \text{ReLU} \\
\downarrow \quad & \text{Linear}(64, 5) \to \text{residual} + \text{constraints} \to \theta^{(b)} \in \mathbb{R}^5
\end{aligned}
$$

The map uses a **residual parameterization** in normalized space: input θ^(a) is z-scored using training-set statistics, the MLP predicts a small delta (scaled by 0.3), and the output is denormalized to target-asset parameter space. This ensures the map starts near identity and learns deviations. Output constraints: σ, λ, σ_J are passed through softplus to guarantee positivity; μ and μ_J are unconstrained.

Weak target model (SimpleJDModel). A single-layer linear model (14 features → 5 JD parameters) trained on each target asset with MAP prior (`prior_weight = 0.1`). This produces a baseline θ^(b) sequence that anchors Phase 3 training — without it, the transfer map has no signal about what target-asset parameters should look like.

6.2 Phase 3: Joint Training

Phase 3 learns the transfer map T by jointly minimizing three loss terms (Eq. 47):

$$L_{\text{unified}} = L_{JD} + \lambda_{TW} \cdot L_{\text{TransWeave}} + \lambda_{wpt} \cdot L_{\text{weakness}}$$

L_{JD} (primary): NLL + CRPS of the mapped parameters against target returns.

$$L_{JD} = \text{NLL}_{\text{clamped}}(r^{(b)},\, T(\theta^{(a)})) + \alpha \cdot \text{CRPS}(T(\theta^{(a)}),\, r^{(b)})$$

NLL is clamped at mean ± 5σ per batch to prevent extreme bars from dominating gradients. $\alpha = 1.0$.

$L_{\text{TransWeave}}$ (structure, Eq. 49): The paper's Bellman–Darboux intertwining condition is approximated by **per-parameter-normalized MSE** $\|(T(\theta^{(a)}) - \theta^{(b)}_{\text{weak}})/\sigma^{(b)}\|^2$, where $\sigma^{(b)}$ is the per-dimension std of $\theta^{(b)}$ on the training set. Raw MSE would let λ (scale ~200) swamp σ_J (scale ~0.02); normalization balances dimensions. We add $\lambda_{\text{ent}} \cdot \text{Ent}(T)$ ($\lambda_{\text{ent}} = 0.01$): an **isometry-based entropy** that penalizes collapse — for batch pairs (a_i, a_j), if $\|T(a_i) - T(a_j)\|/\|a_i - a_j\| < 0.5$, the term increases. This prevents the map from mapping disparate inputs to nearly identical outputs.

L_{weakness} (behavioral, Eq. 50): $-\log W_{PT}(T(\theta^{(a)})) + \text{MSE}(W_{PT}^{\text{pred}}, W_{PT}^{\text{target}})$. The first term encourages behavioral robustness. The second aligns the **weakness profile** of the mapped output with the weak target: we want W_{PT} values to match in magnitude, not just the JD parameters. Predicted and target raw W_{PT} are jointly normalized (batch-max) before MSE for cross-asset comparability.

Phase 3 hyperparameters: lr = 1e-3, batch_size = 2048, patience = 10 on val_unified.

6.3 Full vs Partial vs Weak Modes

The three transfer modes share the same architecture and loss formulas — they differ **only in the λ_{TW} weight** and Phase 4 learning rate:

Hyperparameter	Full	Partial	Weak
λ_{TW} (Phase 3)	0.10	0.05	0.025
Phase 4 lr	5e-5	1e-5	5e-5
Phase 4 grad clip	1.0	0.1	1.0
λ_{wpt} (Phase 3)	0.1	0.1	0.1
λ_{wpt} (Phase 4)	1.0	1.0	1.0

Full mode uses a higher λ_{TW}, pulling $T(\theta^{(a)})$ closer to the weak model's $\theta^{(b)}$. **Partial** mode ($\rho \in [0.5, 0.7]$) gives T more freedom to deviate; with lower λ_{TW}, Phase 4 fine-tuning can induce gradient conflict between L_{JD} and the weakness term when structure is only partially aligned. We use a more conservative Phase 4 lr (1e-5) and tighter grad clip (0.1) to stabilize updates. **Weak** mode ($\rho < 0.5$) uses full lr (5e-5) again: regime overlap is low, so Phase 3's weak-model anchor is less reliable; Phase 4 needs room to adapt to target returns. The L2 anchor still limits drift.

6.4 Phase 4: Fine-tuning

Phase 4 fine-tunes the transfer map on target data, dropping $L_{\mathrm{TransWeave}}$ and simplifying L_{weakness}:

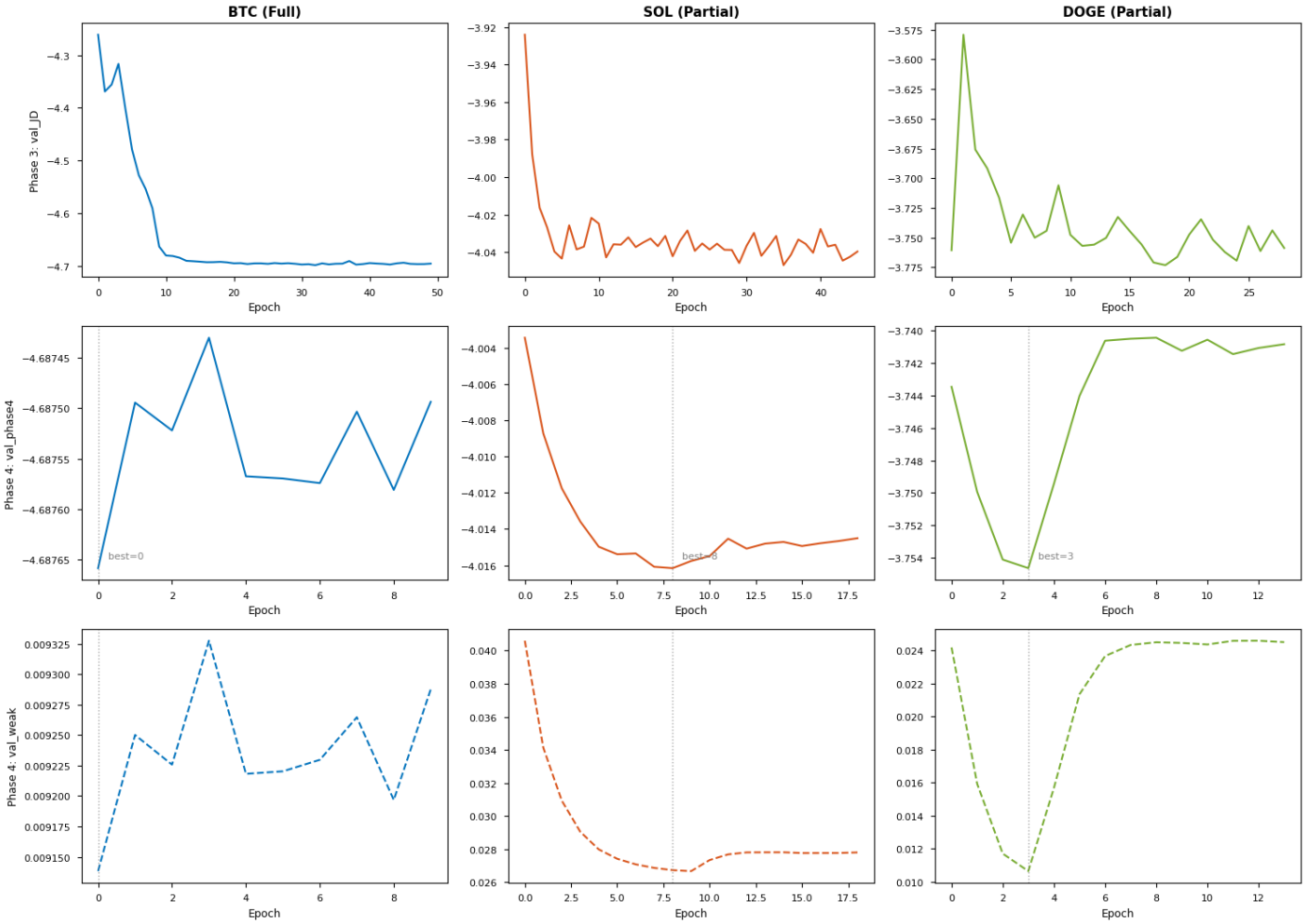
$$L_{\text{Phase 4}} = L_{JD} + \lambda_{wpt} \cdot (-\log W_{PT}) + \mu \cdot \|T - T_{\text{Phase 3}}\|^2$$

Three changes from Phase 3: (1) no BD intertwining constraint — the map is free to adapt beyond the weak model's guidance; (2) L_{weakness} reduces to just $-\log W_{PT}$ (the MSE consistency term is dropped since Phase 3 already aligned the profiles); (3) an **L2 anchor** (μ = 50.0) prevents the map from drifting too far from the Phase 3 solution. Note: the L2 anchor is our implementation choice, not prescribed in the paper — we found it necessary to stabilize $\mathrm{Phase}\ 4$ training, as without it the map can overfit to noisy target returns.

Phase 3/4 interaction lesson. An early implementation used $\lambda_{wpt} = 0.5$ in Phase 3, which pushed W_{PT} to a sharp minimum. Phase 4 then had no room to improve — val_phase4 never beat the Phase 3 baseline. Reducing Phase 3 λ_{wpt} to 0.1 flattens the W_{PT} landscape and allows Phase 4 to make meaningful updates.

As Figure 5 shows, Phase 4 converges quickly — the best epoch is typically within the first few (BTC/DOGE best at epoch 0–3, SOL at epoch 7); beyond that, validation loss plateaus or increases, consistent with overfitting.

Figure 5: TransWeave Training Dynamics



6.5 Per-Target Analysis

BTC (Full mode). Phase 3 runs all 50 epochs with steady val_JD improvement (−4.26 → −4.70). The L_TransWeave term drops from ~1000 to ~1.0 over the first 10 epochs as the map learns the parameter-space alignment. Phase 4 converges immediately (best epoch = 0) — the Phase 3 solution is already near-optimal for BTC, and the L2 anchor prevents meaningful drift. Final test NLL: −4.860 (vs Scratch −4.788, Δ = −0.072). BTC is the clearest transfer success.

SOL (Partial mode). Phase 3 converges in 46 epochs. Phase 4 shows gradual improvement over 8 epochs (val_phase4: −4.003 → −4.016), with val_weak dropping from 0.041 to 0.027 (−34%) — the weakness regularization is actively reshaping the output distribution. Despite this improvement, the final test NLL (−4.142) slightly underperforms Scratch (−4.177, Δ = +0.035). SOL's distinct volatility regime — σ = 0.834 annualized, nearly 2× ETH's — limits how much the ETH-anchored transfer map can help.

DOGE (Partial mode). Phase 3 converges early (29 epochs). Phase 4 best at epoch 3 — the map improves briefly then the L2 anchor pulls it back. Final test NLL: −4.084 (vs Scratch −4.038, Δ = −0.046). DOGE is the most surprising success: despite the lowest ρ_{regime} (0.662), TransWeave matches the fully-trained Target-TFT (−4.083 vs −4.084). This suggests the transfer map captures useful cross-asset structure even for meme-driven assets.

6.6 Direct Transfer Baseline

"Direct" applies the source ETH TFT-JD parameters to target returns without any adaptation — effectively $T = \text{identity}$. It consistently underperforms all other methods (BTC: −4.746, SOL: −4.041, DOGE: −3.850), confirming that naive parameter sharing fails. The gap between Direct and TransWeave (BTC: −0.114, SOL: −0.101, DOGE: −0.234) quantifies the value of the learned mapping.

6.7 Implementation Note: Parameter-Space vs State-Space

The paper's theoretical framework (Section 3) formulates TransWeave using Bellman–Darboux intertwining operators that act on the state-space dynamics. Our implementation simplifies this to a **parameter-space regression**: $T(\theta_t^{(a)}, \text{regime}_t) \to \theta_t^{(b)}$. This means the transfer map sees only the source JD parameters and regime label at each timestep — it does not condition on target-asset features $X_t^{(b)}$.

This is a deliberate simplification. The parameter-space formulation is tractable (the map has only 9 input dimensions), avoids the need for paired source-target feature windows, and still achieves meaningful transfer gains. However, it may limit performance on targets where the source features are uninformative about target dynamics — which may partly explain SOL's underperformance.

---

7. Parameter Dynamics & Diagnostics

7.1 Time-Varying Parameters

The TFT-JD source model produces per-bar JD parameters that should respond to market conditions. We examine the selected source checkpoint's (`eth_tft_jd_w_finetune.ckpt`) five output parameters on the ETH test set, followed by the TransWeave-mapped parameters for each target asset.

Figure 6: ETH Source Model — All 5 JD Parameters (Test Set)

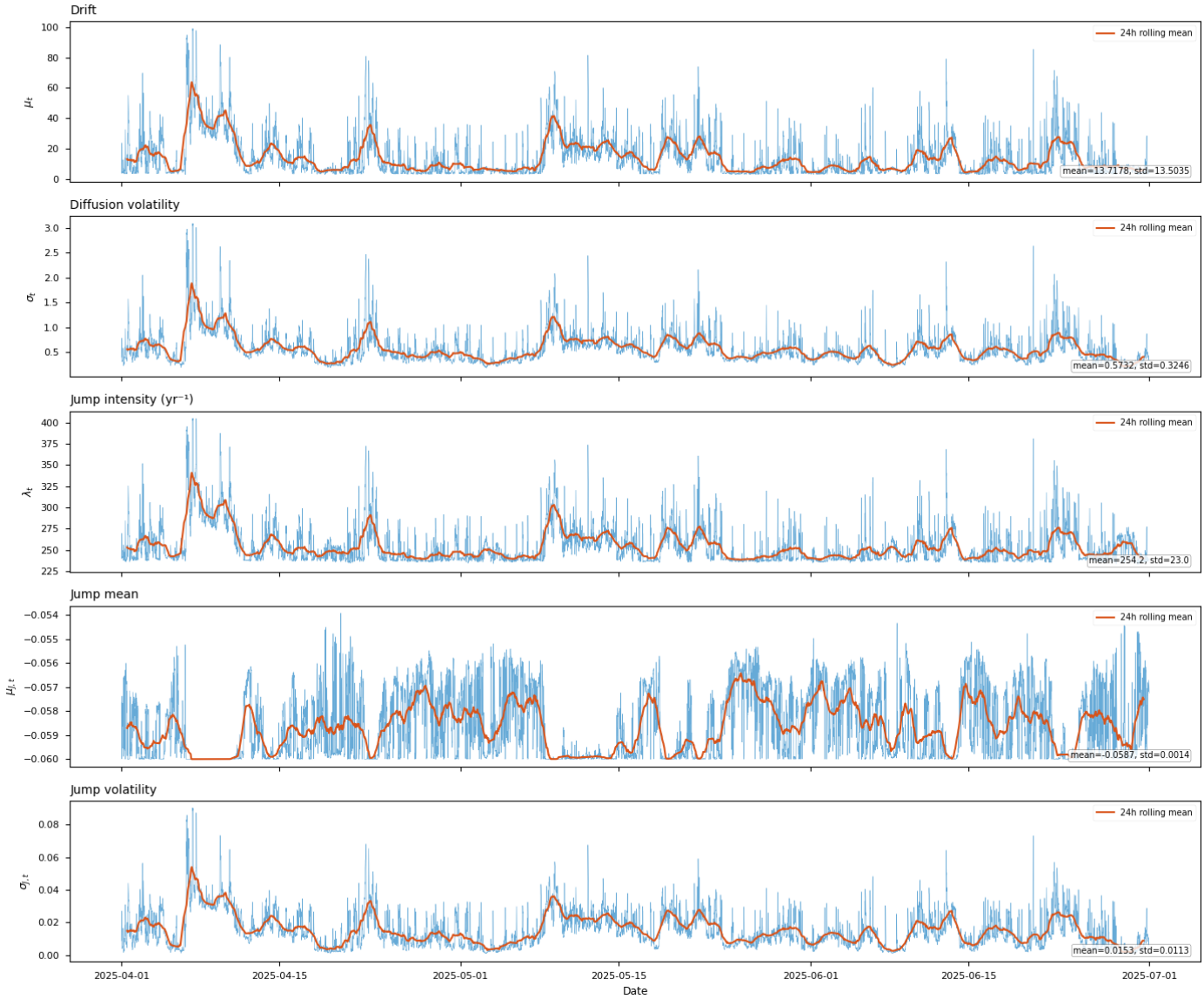
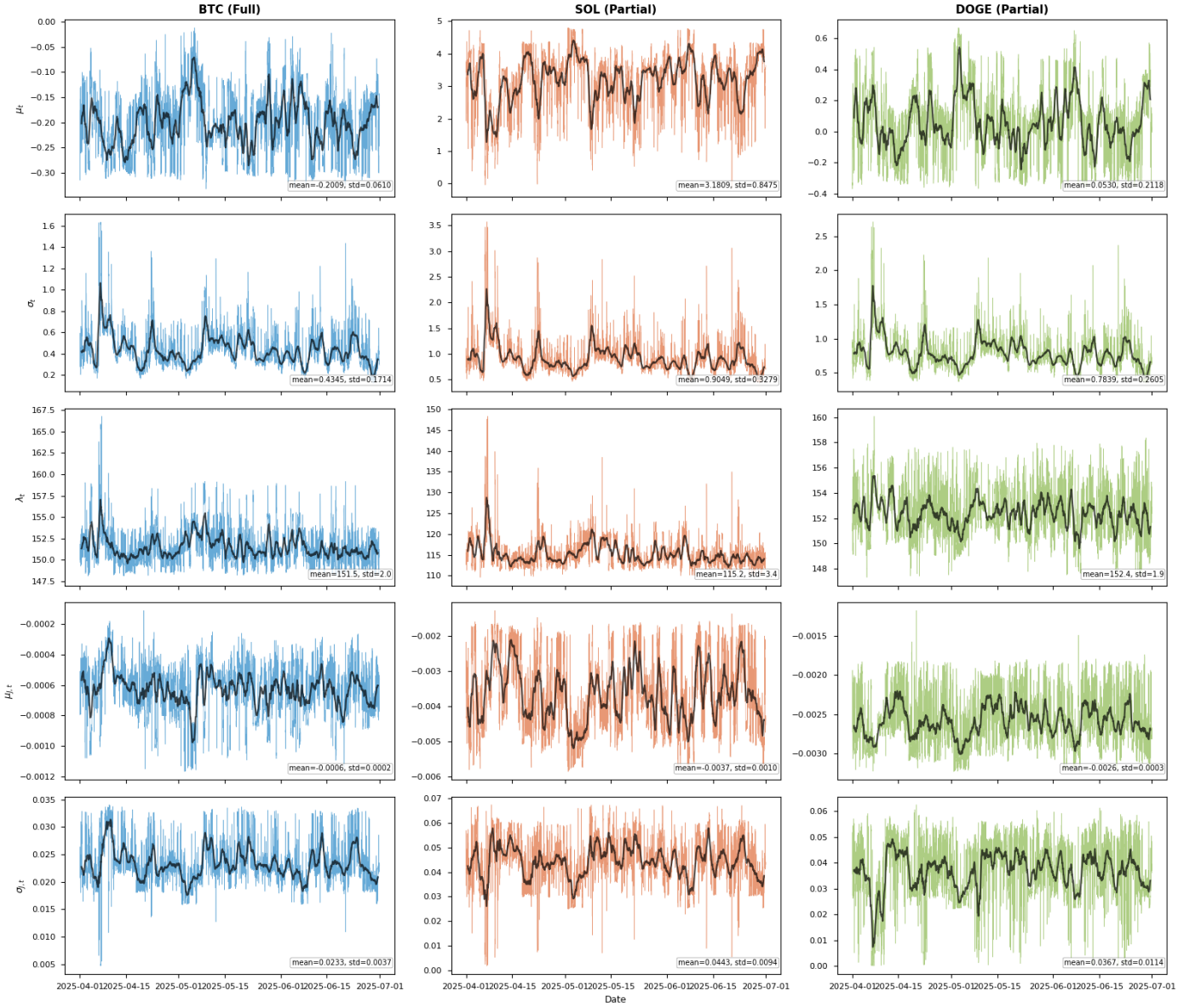


Figure 7: TransWeave-Mapped Parameters for Target Assets (Test Set)



7.2 Diagnostic Observations

ETH source model (Figure 6):

- σ_t **shows the most variation.** Diffusion volatility tracks market conditions well — spikes align with major price moves and elevated realized volatility.
- λ_t **has limited variation.** Despite the W-finetune model having more λ variation than the base model (std ≈ 23 vs 0.5), the coefficient of variation is still modest (~9%). The model prefers adjusting σ_t and $\sigma_{J,t}$ to fit changing conditions.
- μ_t **and** $\mu_{J,t}$ **cluster near zero.** Drift is small relative to volatility at 15-minute resolution, and the jump mean is slightly negative, reflecting the well-documented asymmetry of crypto jumps.

Target asset parameters (Figure 7):

The TransWeave-mapped parameters show how the transfer map adapts ETH's parameter dynamics to each target. Key observations:

- **BTC** parameters closely track ETH's shape (same temporal patterns) with modest level shifts — consistent with Full transfer mode and high ρ.
- **SOL** shows larger σ_t values (reflecting its higher realized volatility) and more divergent λ_t dynamics.
- **DOGE** parameters exhibit the most distinct levels but still preserve ETH's temporal structure through the regime conditioning.

7.3 Transfer Map Output Quality

For the transfer experiments, a key diagnostic is whether the mapped parameters $T(\theta^{(a)})$ produce well-calibrated return distributions on the target asset. We assess this through the full NLL + CRPS comparison:

Table 5. Complete test-set metrics (single-line format for PDF export):

Target	Mode	Scratch NLL	TW NLL	Target-TFT NLL	Scratch CRPS	TW CRPS	Target-TFT CRPS
BTC	Full	-4.788	-4.860	-4.888	0.00113	0.00110	0.00108
SOL	Partial	-4.177	-4.142	-4.211	0.00217	0.00220	0.00213
DOGE	Partial	-4.038	-4.084	-4.083	0.00245	0.00241	0.00239

NLL and CRPS tell the same story: TransWeave improves both metrics for BTC and DOGE, underperforms on SOL. The CRPS improvements are proportionally larger than NLL improvements on DOGE — the transfer map produces a better-calibrated full distribution, not just a higher peak density.

For DOGE, TransWeave's CRPS (0.00241) is within a negligible margin of the target-only benchmark (Target-TFT CRPS 0.00239). This is notable because DOGE has the lowest regime overlap with ETH — the transfer map generalizes better than the raw ρ_{regime} metric would predict.

---

8. Limitations & Future Work

8.1 Known Limitations

1. Parameter-space simplification. Our TransferMap operates in JD parameter space rather than the state-space Bellman–Darboux formulation described in the paper (Section 3). This design choice was motivated by computational tractability: a parameter-space regression $T(\theta^{(a)}, \text{regime}) \to \theta^{(b)}$ is straightforward to optimize, while a full state-space implementation would require paired feature inputs and additional architectural complexity. The trade-off is that the map cannot condition on target-asset features $X^{(b)}$ and relies entirely on source parameters and regime labels. A state-space extension could potentially improve performance on dissimilar targets like SOL.

2. Regime conditioning uses only source (ETH) regime. The paper's formula (Eq. 22) $T = V^{(b)}\Pi(V^{(a)})^{-1}$ implies that both source and target regime structure should inform the transfer map. Our implementation conditions T only on ETH's regime; target regimes exist from Stage 5 but are not used as input. Intuitively, conditioning on target regime (e.g. "when SOL is in high-volatility regime") could improve mapping. This is a known gap relative to the paper's regime-alignment formulation.

3. Limited asset coverage. We test transfer on only three target assets, all cryptocurrencies. While BTC/SOL/DOGE span a range of market dynamics, three targets is a small sample for validating the transfer framework's generality. A larger-scale study — covering 10+ crypto assets, or extending to equities and forex — would better demonstrate the paper's potential for systematic cross-asset modeling.

4. Feature set could be richer. Our 14-feature input covers standard price-volume indicators and 4 on-chain metrics. The TFT is capable of ingesting many more features — order book depth, funding rates, cross-exchange spreads, sentiment indicators — that could improve both source model quality and transfer effectiveness. The current feature set was chosen for reproducibility, not optimality.

5. Hyperparameter tuning was manual. Key hyperparameters (λ_{wpt}, Phase 4 lr, L2 anchor weight) were tuned by manual iteration rather than systematic search (grid search, Bayesian optimization). The Phase 3/4 interaction (§6.4) was particularly sensitive — a more rigorous search over the joint ($\lambda_{wpt}^{\text{P3}}$, $\lambda_{wpt}^{\text{P4}}$, lr) space could yield better results, especially for the underperforming SOL target.

---

9. Conclusion

We have reproduced the core pipeline of Goertzel (2025) using cryptocurrency data: a Merton jump-diffusion model with time-varying parameters conditioned via a Temporal Fusion Transformer, a prospect-theory weakness regularizer, HMM-based regime detection with multi-factor transfer feasibility scoring, and a parameter-space TransWeave transfer map.

Key findings:

1. **TFT conditioning is the dominant contributor** to source model quality. The jump from Static-JD to TFT-JD (ΔNLL = −0.19) is 5× larger than adding the jump component itself (GBM → Static-JD, ΔNLL = −0.04). Time-varying parameters matter more than the parametric specification.

2. **TransWeave transfer works for 2 of 3 targets.** BTC (Δ = −0.072) and DOGE (Δ = −0.046) benefit from transfer; SOL (Δ = +0.035) does not. The composite S_{transfer} score, computed before any training, correctly predicts this ordering.

3. **Unconstrained MLE pushes λ to degenerate values** ($\lambda > 9000$ with $\sigma_J \to 0$). The empirical MAP prior resolves this for the static model; the W-finetune training mode further enables meaningful temporal variation in λ_t — an effect that neither the prior alone nor weakness-from-scratch achieves.

4. **The decision framework works.** SOL's $\rho_{\text{regime}} = 0.714$ passes the Full threshold, but its $S_{\text{transfer}} = 0.391$ correctly triggers a downgrade to Partial mode, reflecting the genuine difficulty of transferring to an asset with 2× higher volatility.

This empirical implementation provides supportive evidence for the paper's central thesis: cross-asset transfer in parameter space is feasible when source and target share sufficient regime structure, and the multi-factor feasibility framework is directionally informative about which transfers will succeed in this three-target experiment.